



Fase 7 — Machine Learning para Detecção de Anomalias

Nesta fase criei um modelo de machine learning que detecta padrões suspeitos nas contas financeiras, indo além das regras fixas da Fase 5.

A ideia é usar **Isolation Forest**, um algoritmo não-supervisionado que identifica outliers (valores atípicos) sem precisar de dados rotulados.



Objetivo da Fase

- Treinar um modelo de ML para detectar anomalias automaticamente
 - Identificar padrões suspeitos que não seguem regras simples
 - Complementar as regras de auditoria com detecção baseada em aprendizado
-



Tecnologias Utilizadas

- **Python 3.x**
 - **scikit-learn** — biblioteca de machine learning
 - **Isolation Forest** — algoritmo de detecção de anomalias
 - **Pandas** — manipulação de dados
 - **joblib** — serialização do modelo treinado
-



Por que Isolation Forest?

O **Isolation Forest** é ideal para este caso porque:

1. **Não precisa de dados rotulados** — não temos exemplos marcados de “fraude” vs “normal”
2. **Detecta outliers multidimensionais** — considera várias features ao mesmo tempo
3. **Rápido e eficiente** — funciona bem mesmo com datasets pequenos
4. **Funciona em qualquer distribuição** — não assume que os dados são normais

A ideia do algoritmo é simples: anomalias são mais fáceis de isolar que dados normais. Se um ponto precisa de poucas divisões para ser isolado, provavelmente é uma anomalia.



Features Utilizadas

O modelo analisa 6 características de cada conta paga:

Feature	Descrição
valor_original	Valor da conta
diferenca_valor	Diferença entre valor original e pago (R\$)
perc_diferenca	Percentual de diferença
dias_atraso	Dias entre vencimento e pagamento
desvio_da_media	Quanto o valor se afasta da média da empresa
tipo_binario	1 = A Receber, 0 = A Pagar

Essas features capturam:

- **Valores absolutos** (quanto foi pago)
- **Divergências** (se pagou diferente do esperado)
- **Comportamento temporal** (se atrasou)
- **Contexto da empresa** (se está fora do padrão dela)
- **Tipo da operação** (receita ou despesa)

Arquitetura do Pipeline

1. Extração → Busca contas pagas (MySQL ou CSVs como fallback)
2. Feature Engineering → Cria as 6 features numéricas
3. Normalização → StandardScaler para padronizar escalas
4. Treinamento → Isolation Forest com 100 árvores
5. Predição → Score de anomalia para cada conta
6. Relatório → Top anomalias salvas em CSV

Como Executar

1. Treinar o modelo

```
python src/ml/detector_anomalias.py
```

Saída esperada

```
=====
FiscalAudit AI – Detector de Anomalias (ML)
=====

1. Extraíndo dados...
  ✓ 188 contas pagas carregadas

2. Criando features...
  ✓ Features criadas

3. Treinando modelo Isolation Forest...
  ✓ Modelo treinado com 188 registros
  ✓ Features utilizadas: valor_original, diferenca_valor, perc_diferenca, dias_atraso,
  desvio_da_media, tipo_binario

4. Detectando anomalias...
  ⚠ 19 anomalias detectadas (10.1% do total)

5. Gerando relatório...
  ✓ Relatório salvo em anomalias_ml.csv

=====
Top 5 Anomalias Mais Suspeitas:
=====

• Conta #330 – Albuquerque
  Tipo: A Receber
  Valor: R$ 46,645.60 → Pago: R$ 39,768.61
  Diferença: R$ 6,876.99 (14.7%)
  Atraso: 12 dias
  Score de anomalia: -0.103
```

Arquivos Gerados

Modelo Treinado

- `models/isolation_forest.pkl` — modelo serializado (pode ser reutilizado)
- `models/scaler.pkl` — normalizador (necessário para predições futuras)

Relatório

- `data/processed/anomalias_ml.csv` — lista de anomalias detectadas com scores

Resultados

Rodando contra os dados da Fase 3 (188 contas pagas):

- **19 anomalias detectadas** (~10% do total)
- **Score médio das anomalias:** -0.06 (quanto mais negativo, mais suspeito)
- **Top anomalia:** Conta #330 com score -0.103

Comparação com Regras Fixas (Fase 5)

Método	Divergências Encontradas
Regras Fixas (Fase 5)	23 contas com <code>valor_pago ≠ valor_original</code>
Machine Learning (Fase 7)	19 anomalias (considerando múltiplas features)

O ML detectou um subconjunto das divergências, mas considerando o **contexto completo** (não só se pagou diferente, mas quanto, quando, e se está fora do padrão da empresa).



Interpretando o Score de Anomalia

O **score de anomalia** vai de positivo (normal) a negativo (anômalo):

- **Score > 0:** comportamento normal
- **Score ≈ 0:** fronteira entre normal e anômalo
- **Score < -0.05:** suspeito, merece atenção
- **Score < -0.10:** muito anômalo, revisar com prioridade



Vantagens do Modelo de ML

Sobre Regras Fixas

1. **Considera múltiplas dimensões** — não é só “pagou diferente”, é “pagou diferente + atrasou + valor alto + fora do padrão”
2. **Aprende o que é normal** — cada empresa tem seu padrão, o modelo aprende isso
3. **Detecta combinações suspeitas** — pode achar casos que nenhuma regra individual pegaria

Limitações

1. **Precisa de dados** — funciona melhor com mais histórico
2. **Não explica diretamente** — diz “é anômalo” mas não diz exatamente por quê
3. **Pode dar falsos positivos** — valores legítimos mas incomuns podem ser marcados

Por isso o ideal é usar **ML + Regras** juntos.



Reutilizando o Modelo

Para usar o modelo treinado em novos dados:

```
import joblib
import pandas as pd

# carrega modelo e scaler
modelo = joblib.load('models/isolation_forest.pkl')
scaler = joblib.load('models/scaler.pkl')

# prepara novos dados (mesmas features)
novos_dados = pd.DataFrame({
    'valor_original': [50000],
    'diferenca_valor': [5000],
    'perc_diferenca': [10],
    'dias_atraso': [15],
    'desvio_da_media': [20],
    'tipo_binario': [1]
})

# normaliza e prediz
X_scaled = scaler.transform(novos_dados)
predicao = modelo.predict(X_scaled) # -1 = anomalia, 1 = normal
score = modelo.decision_function(X_scaled)

print(f"Predição: {'ANOMALIA' if predicao[0] == -1 else 'NORMAL'}")
print(f"Score: {score[0]:.3f}")
```

Próximos Passos

Com o modelo de ML treinado, as próximas fases podem incluir:

- **Fase 8** — IA Generativa para gerar relatórios em linguagem natural explicando as anomalias
- **Fase 9** — Dashboard interativo com Streamlit para visualizar tudo
- **Fase 10** — Docker e documentação final para deploy